



第5回 Pythonによるデータ可視化の基礎 (Matplotlib)

講師・スライド作成 中内

2つの教材を用いますが、データ可視化と単回帰分析に必要な話だけ解説します

Pythonによるデータ可視化の基礎 (Matplotlib) .ipynb

- 1.1 データの可視化
- 1.2 データ可視化の基礎
- Appendix: EDAを自動化する様々なライブラリ

統計・確率.ipynb ---> こちらの教材もあくまでデータ可視化の話のみに触れます

- 2.3.4 要約統計量とパーセンタイル値
- 2.3.5 箱ひげ図

→ 箱ひげ図の理解

- 2.3.7 散布図と相関係数

→ ヒートマップの理解

- 2.4 単回帰分析

→ ※このトピックだけ少し別テーマ（機械学習の入り口の話）

上記以外の箇所はGCI修了後に更なるスキルアップを目指す際に役立ててください

データ可視化

- 数値を**グラフ**で表す
- そこから考察
- 探索的にグラフを改善



線形単回帰分析

- 数値から**未来を予測**
- 実測値の分布に近似する直線を引く



データ可視化

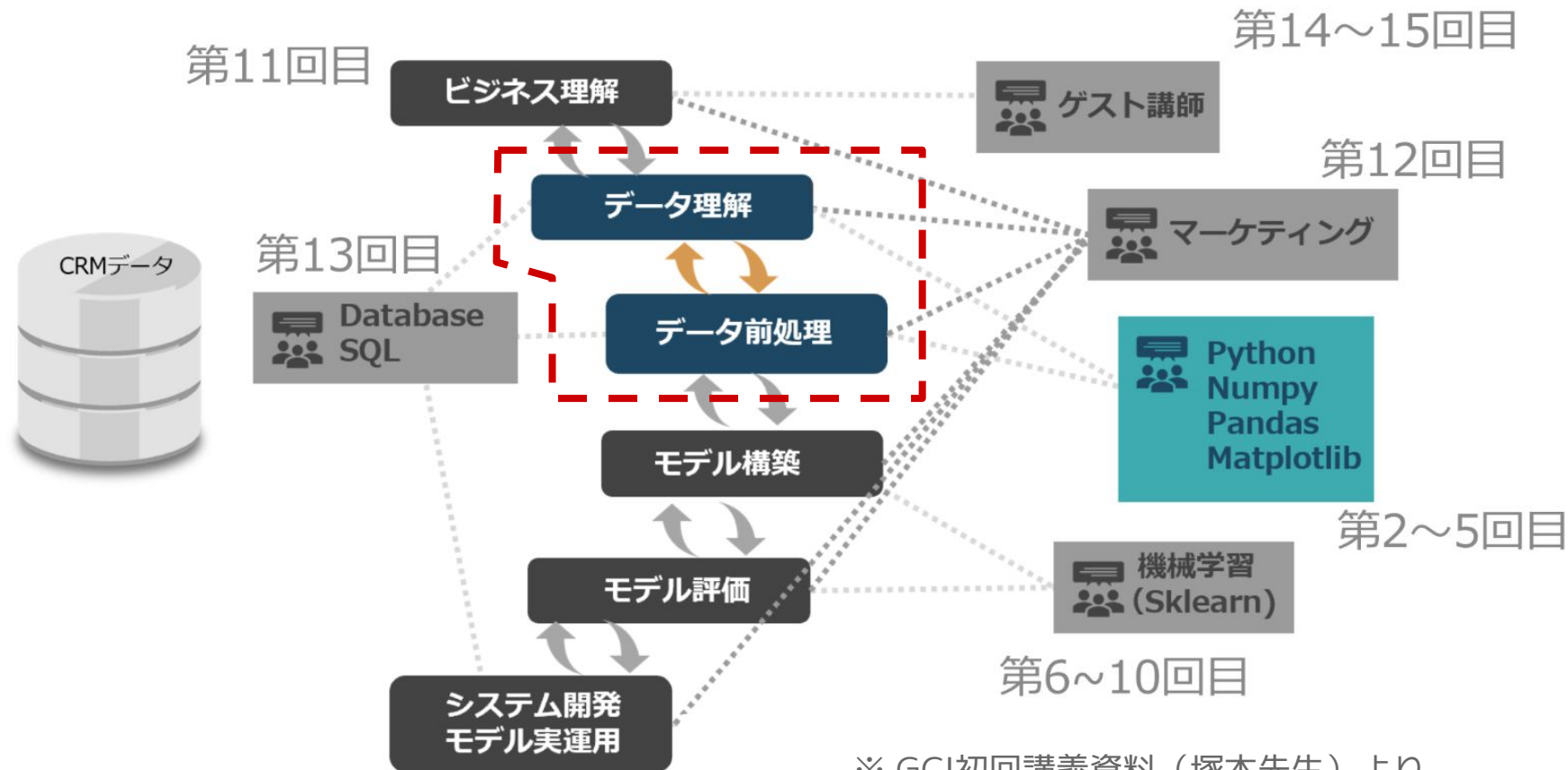
- 数値を**グラフ**で表す
- そこから考察
- 探索的にグラフを改善



線形単回帰分析

- 数値から**未来を予測**
- 実測値の分布に近似する
直線を引く





※ GCI初回講義資料（塚本先生）より

- データセットに**前処理を施しながら**統計量を抽出して**可視化**し
仮説立案と検証を繰り返してデータがもつ**特性・パターン・偏り**を
探索的に明らかにしていくこと
- 探索的であることが重要であり、必然的に**試行錯誤を重ねること**になる
- ①理論やドメイン知識^(※)などの事前知識を活用した仮説立案と
②バイアスを排した特徴観察の両方が重要（知的好奇心も重要）

※ドメイン知識：データが対象としている業務分野（＝ドメイン）についての知識のこと

- データ可視化は**自分自身がEDAを効果的に行うことが第一の目的**
- これとは別に「**コミュニケーションスキルとしての可視化**」という場面もあるが、これと「**EDAとしての可視化**」では若干考え方や必要なスキルセットが異なることに注意すべき

🔍 課題の[発見]に力点（EDA）

（可視化によって…）
クライアントに提起すべき課題
を**発見**したい

サクサクと短時間で可視化するスキル

🗣️ 課題の[伝達]に力点（プレゼン）

（可視化によって…）
クライアントに課題を**伝えたい**

グラフを丁寧に作り込むスキル



フェーズに応じた対応

各フェーズで必要な可視化のスキルは若干異なることへの認識が必要

Pythonにおけるデータ可視化では
主に**Matplotlib**というライブラリを使う



© Copyright The Matplotlib development team.
<https://matplotlib.org/>

matplotlib

【概要】

matplotlibの本体

【特徴】

グラフの各部位を一つひとつ丁寧に定義していくスタイル

【長所】

細かいところまで図を制御できる

【短所】

pyplotに比べて**コードの記述量が多くなる傾向**がある

matplotlib.pyplot

【概要】

状態管理型インターフェース

【特徴】

短いコードで完結させるスタイル

【長所】

標準設定でかまわないところは大胆に省略して短いコードで使える

【短所】

マニアックな作り込みをしたい時には制約が生じることがある

最も手軽には数行でグラフ化できる(matplotlib.pyplot)

① 描画したいデータを用意

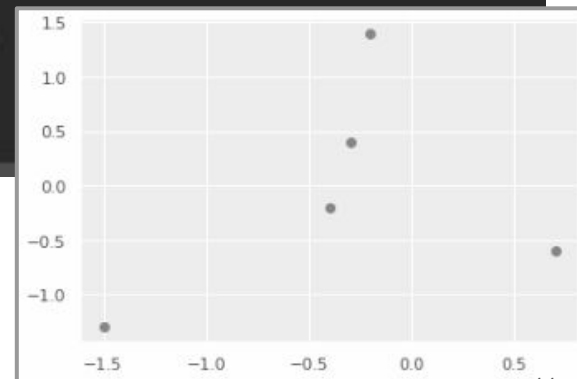
```
# x軸とy軸のそれぞれに表示したいデータ  
x = [-1.5, 0.7, -0.4, -0.2, -0.3]  
y = [-1.3, -0.6, -0.2, 1.4, 0.4]
```

② 各軸にどのデータを表示するか、
グラフの種類は何か、の2点を最小限
指定する

```
# グラフを描画  
plt.plot(x, y, 'o')
```

③ グラフ表示を指示
(GoogleColab では省略可)

```
# グラフを表示  
plt.show()
```



最小限ならたったこれだけで描画できる
→ **グラフ作成は少しも面倒ではない**

最も手軽には数行でグラフ化できる(matplotlib.pyplot)

次のようにモジュールをインポートしておく

```
import matplotlib.pyplot as plt
```

グラフを**描画**する関数

plt.plot()

x軸, y軸のデータを引数に渡す

グラフを**表示**する関数

plt.show()



```
# x軸とy軸のそれぞれに表示したいデータ
```

```
x = [-1.5, 0.7, -0.4, -0.2, -0.3]
```

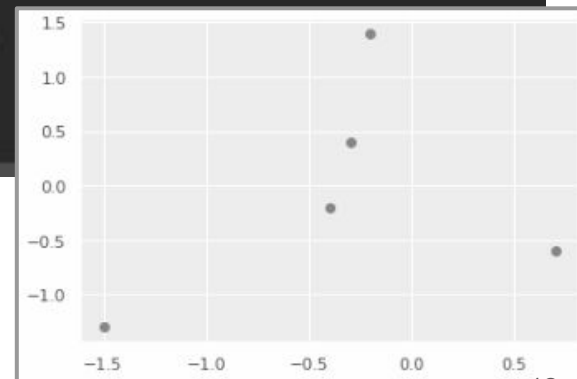
```
y = [-1.3, -0.6, -0.2, 1.4, 0.4]
```

```
# グラフを描画
```

```
plt.plot(x, y, 'o')
```

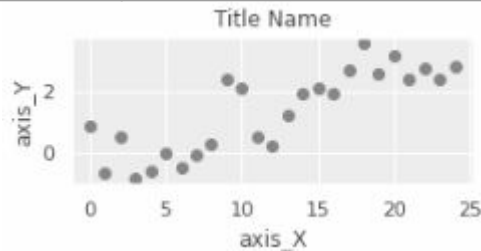
```
# グラフを表示
```

```
plt.show()
```



3つ目の引数に指定する記号
(`'o'`の部分) によってグラフの
マーカーを簡単に変更できる

<code>'o'</code>	マーカー表示
<code>'-'</code>	連続線で表示
	その他数十種類

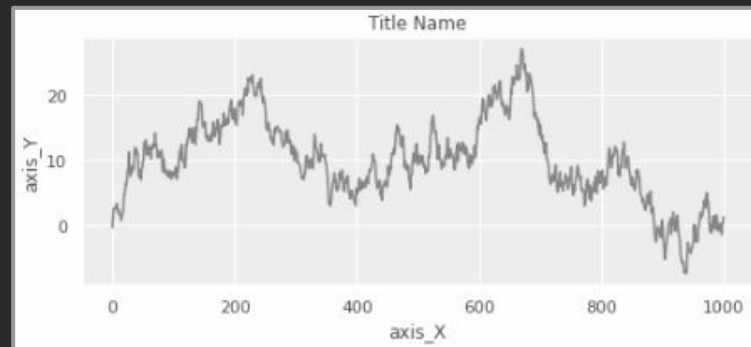


```
# 仮にこんなデータがx, yに入っていたとする  
x = np.arange(1000)  
y = np.random.randn(1000).cumsum()
```

```
# 前のスライドで説明した部分  
plt.title('Title Name')  
plt.xlabel('axis_X')  
plt.ylabel('axis_Y')
```

```
# 3つ目の引数でグラフ種類を変更できる  
plt.plot(x, y, '-')
```

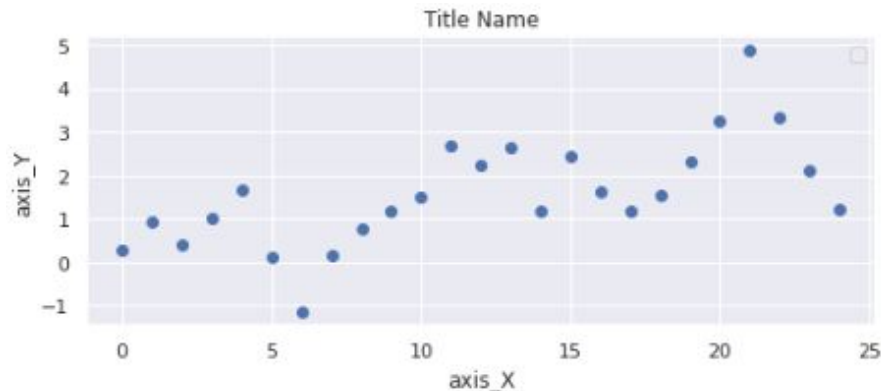
```
plt.show()
```



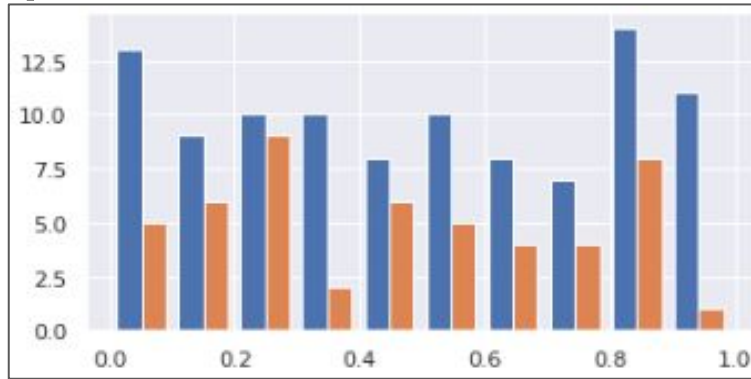
- 前のスライドでは右図①の方法でグラフの種類を指定した
- 更にさまざまなグラフ種類を試したい場合は、②の方法で指定することもできる
- ②の方法の場合、「**scatter**」の部分を経々なグラフの名前に置き換えることで多様なグラフを使い分けられる

① `plt.plot(x, y, 'o')`

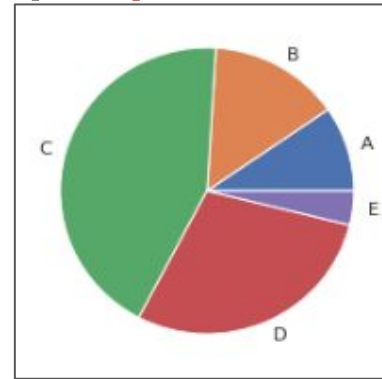
② `plt.scatter(x, y)`



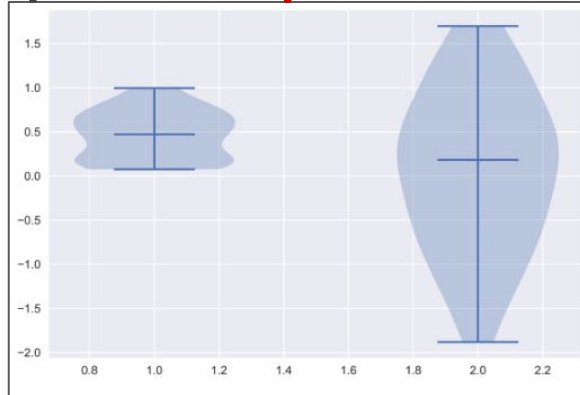
`plt.hist()`



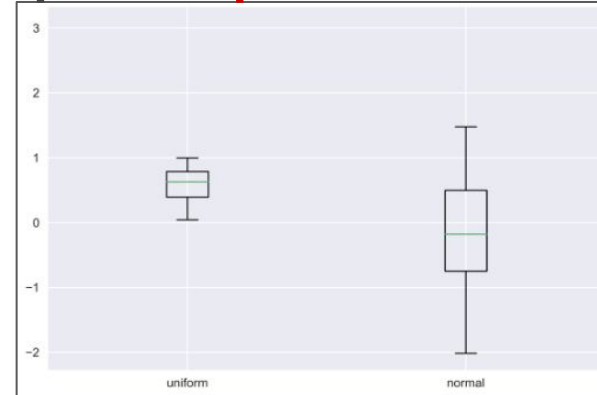
`plt.pie()`



`plt.violinplot()`



`plt.boxplot()`



①
タイトルを追加できる
plt.title()

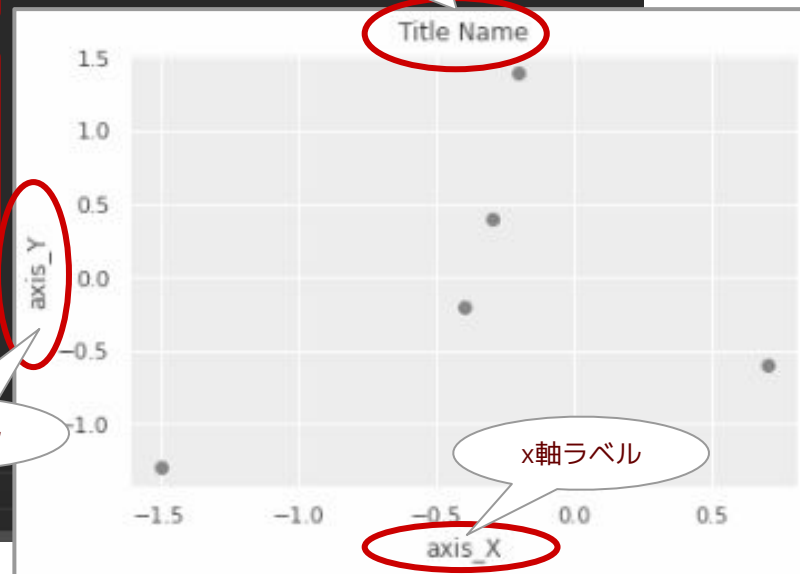
②
X軸とY軸のそれぞれに
ラベルを追加できる
plt.xlabel()
plt.ylabel()

```
# 前のスライドで説明した部分  
x = [-1.5, 0.7, -0.4, -0.2, -0.3]  
y = [-1.3, -0.6, -0.2, 1.4, 0.4]  
plt.plot(x, y, 'o')
```

①
グラフタイトルを追加
plt.title('Title Name')

②
Xの座標名を追加
plt.xlabel('axis_X')
Yの座標名を追加
plt.ylabel('axis_Y')

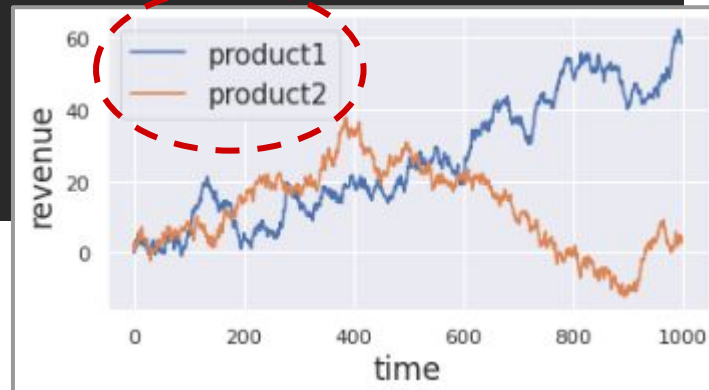
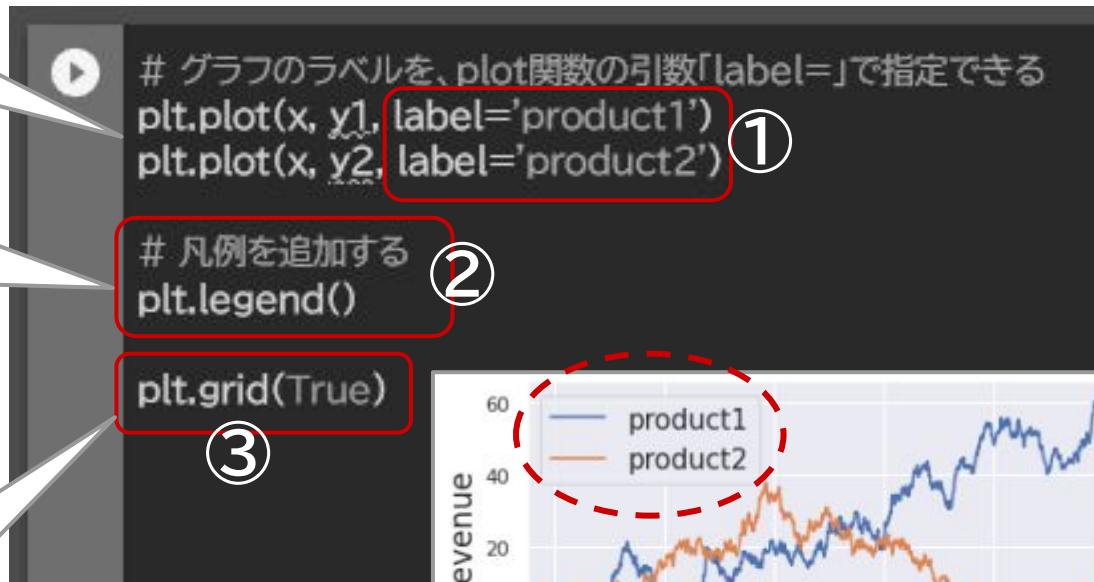
```
plt.show()
```



①plt.plotの引数「label=」で
ラベル名を指定した上で…

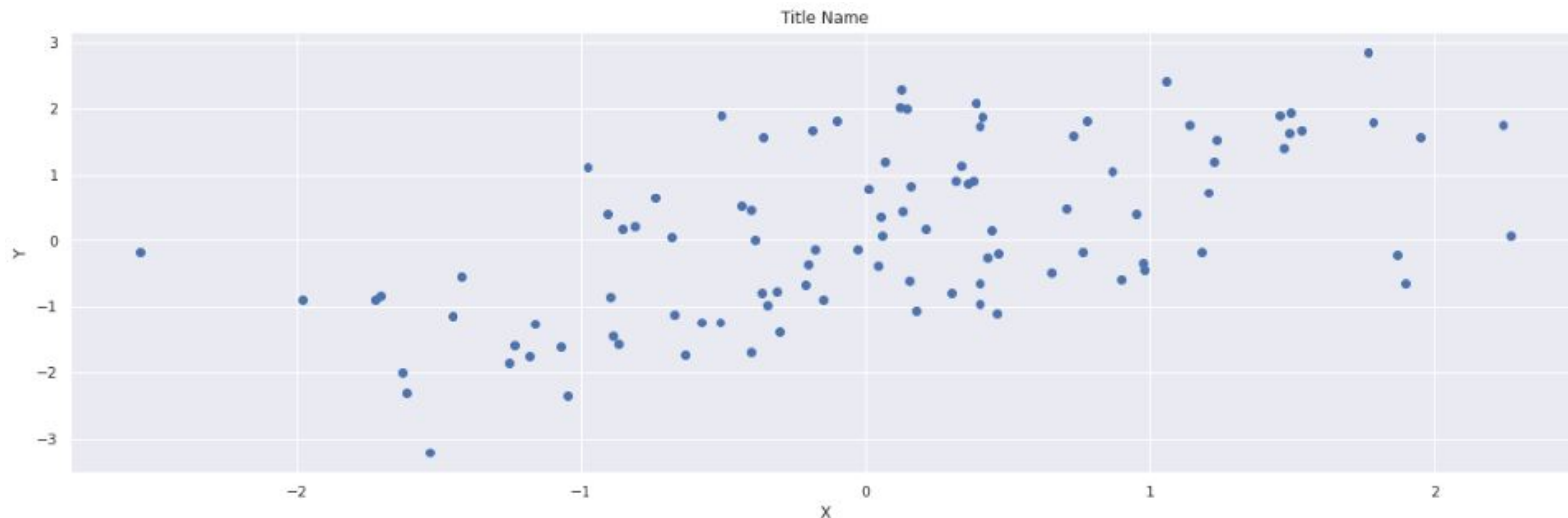
②凡例を追加できる
plt.legend()

③グリッド線を追加できる
plt.grid()
引数はブール値(True or
False)で渡す



```
plt.figure(figsize={x軸の幅}, {y軸の高さ}))
```

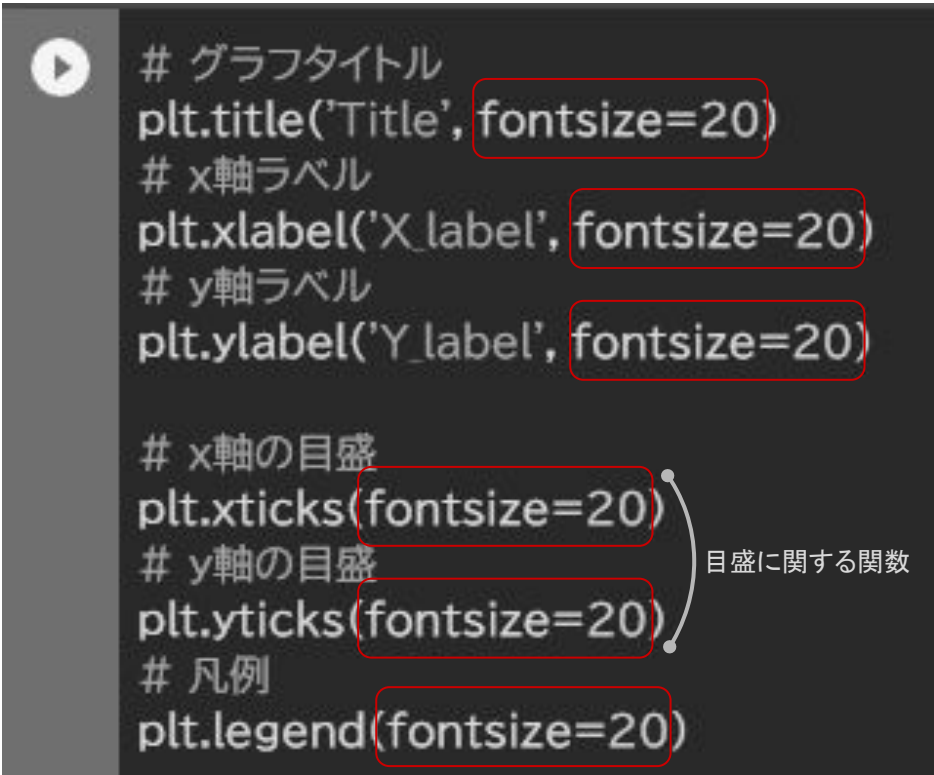
(例) plt.figure(figsize=(20, 6))



各ラベルはいずれも引数に
fontsizeを指定することで文字
の大きさを変更できる

フォントサイズだけではなく、
色、書体、その他細かい調整も
可能

細かい点は覚えるのではなく
調べ方を身につける



```
# グラフタイトル
plt.title('Title', fontsize=20)
# x軸ラベル
plt.xlabel('X_label', fontsize=20)
# y軸ラベル
plt.ylabel('Y_label', fontsize=20)

# x軸の目盛
plt.xticks(fontsize=20)
# y軸の目盛
plt.yticks(fontsize=20)
# 凡例
plt.legend(fontsize=20)
```

目盛に関する関数

The image shows a sequence of three overlapping screenshots of the Matplotlib documentation website, illustrating how to find a specific function.

- Top Screenshot:** The main documentation page for Matplotlib 3.5.1. The "Reference" link in the top navigation bar is circled in red and labeled with a red "1".
- Middle Screenshot:** The "API Reference" page. The left sidebar contains a list of modules. "matplotlib.pyplot" is circled in red and labeled with a red "2".
- Bottom Screenshot:** A closer view of the "matplotlib.pyplot" module page. The "matplotlib.pyplot" link in the sidebar is circled in red and labeled with a red "3".

A large grey arrow on the right side of the images points from the top screenshot down to the bottom screenshot, indicating the flow of navigation.

公式ドキュメントトップページから
「Reference」に入り、ページ左側にある見出しリストから使い方を調べたい関数を探し出す。

引用元：Matplotlib Development Team, Matplotlib Documentation, <https://matplotlib.org/>

引用元 : Matplotlib Development Team, Matplotlib Documentation, <https://matplotlib.org/>

見出し。住所のように「matplotlibの中のpyplotの中のxlabelという関数について」という構成になっている

matplotlib.pyplot.xlabel

```
matplotlib.pyplot.xlabel(xlabel, fontdict=None, labelpad=None, *,  
loc=None, **kwargs)
```

[\[source\]](#)

Set the label for the x-axis.

Parameters:

xlabel : *str*

The label text.

labelpad : *float, default: rcParams["axes.labelpad"] (default: 4.0)*

Spacing in points from the Axes bounding box including ticks and tick labels. If None, the previous value is left as is.

loc : *{'left', 'center', 'right'}, default:*

rcParams["xaxis.labellocation"] (default: 'center')

The label position. This is a high-level alternative for passing parameters *x* and *horizontalalignment*.

Other Parameters: ****kwargs** : *Text properties*

Text properties control the appearance of the label.

どんな引数を指定できるかが()の中に示されている

各引数の詳細説明領域

どんなデータ型で入力すべきかや、どのような選択肢が用意されているかなどが書かれている

その他様々な関数と共通のパラメータ。(フォントサイズのようなラベルの外観に関わるものは**Text properties**を参照するよう記載されている。)

フォントを調整するための共通の
パラメータが全て記載されている

(膨大な項目数がある)

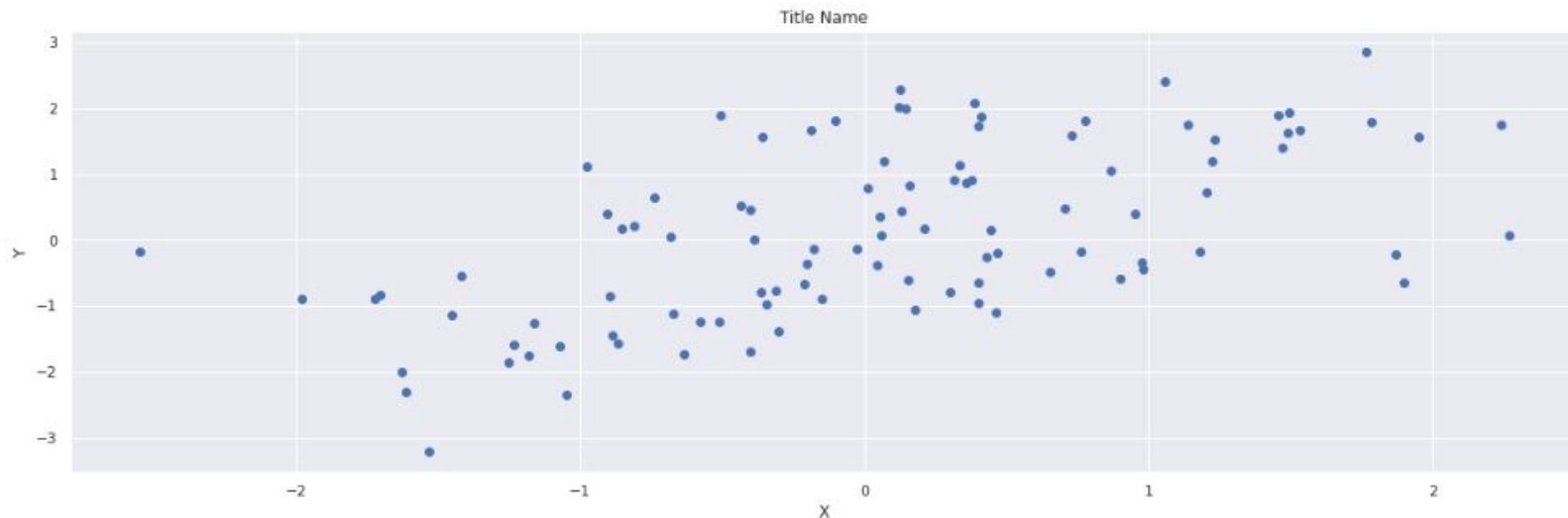
例えば前のスライドで示した引数の
「fontsize」はここに説明を見つける
ことができる

Property	Description
<code>agg_filter</code>	a filter function, which takes a (m, n, 3) float array and a dpi value, and returns a (m, n, 3) array
<code>alpha</code>	scalar or None
<code>animated</code>	bool
<code>backgroundcolor</code>	color
<code>bbox</code>	dict with properties for <code>patches.FancyBboxPatch</code>
<code>fontfamily</code> or <code>family</code>	{ <code>FONTNAME</code> , 'serif', 'sans-serif', 'cursive', 'fantasy', 'monospace'}
<code>fontproperties</code> or <code>font</code> or <code>font_properties</code>	<code>font_manager.FontProperties</code> or <code>str</code> or <code>pathlib.Path</code>
<code>fontsize</code> or <code>size</code>	float or {'xx-small', 'x-small', 'small', 'medium', 'large', 'x-large', 'xx-large'}

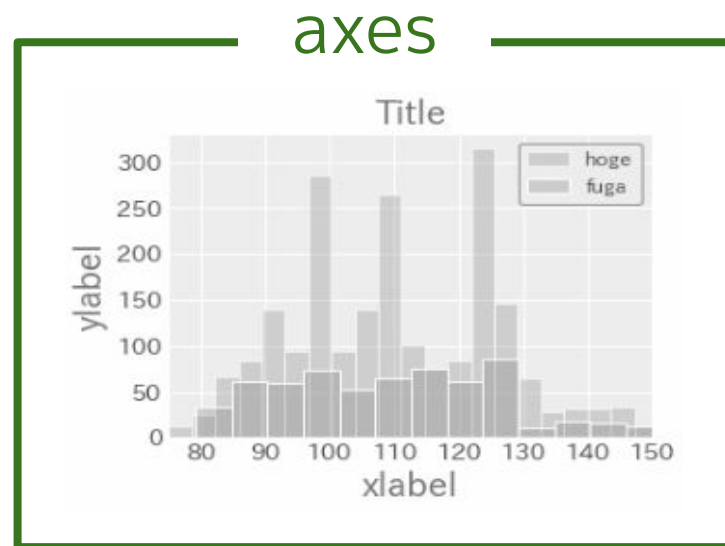
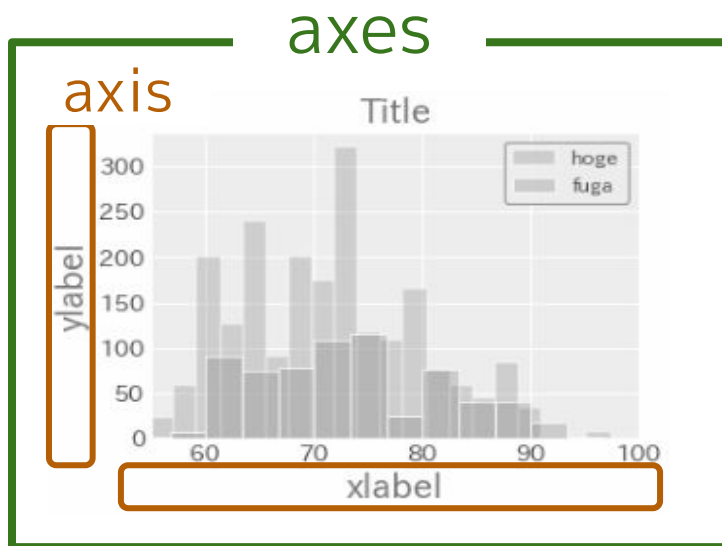
引用元 : Matplotlib Development Team,
Matplotlib Documentation,
https://matplotlib.org/stable/api/text_api.html#matplotlib.text.Text

```
plt.figure(figsize={x軸の幅}, {y軸の高さ}))
```

(例) plt.figure(figsize=(20, 6))



figure



複数のaxesを持つグラフ

先ほどはグラフサイズの指定という文脈で紹介したが、この関数は「Figureの新規作成」が本来の機能（1つしかFigureを作らない場合は省略可）

Figureのなかに複数のAxesを作成する関数
カッコの中に3つの引数が並んでいる。意味は次のとおり。

`plt.subplot( 2, 1, 2 )`

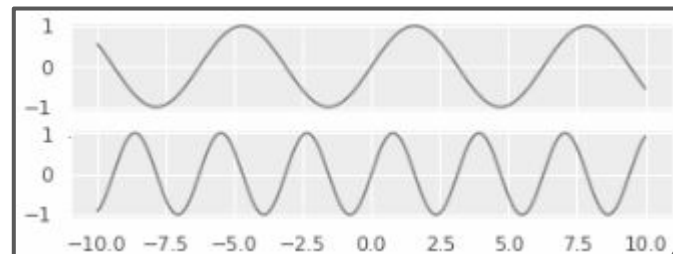
縦に2行、横に1列のAxesのうちの2つ目

```
# グラフの大きさを指定
① plt.figure(figsize=(20, 6))

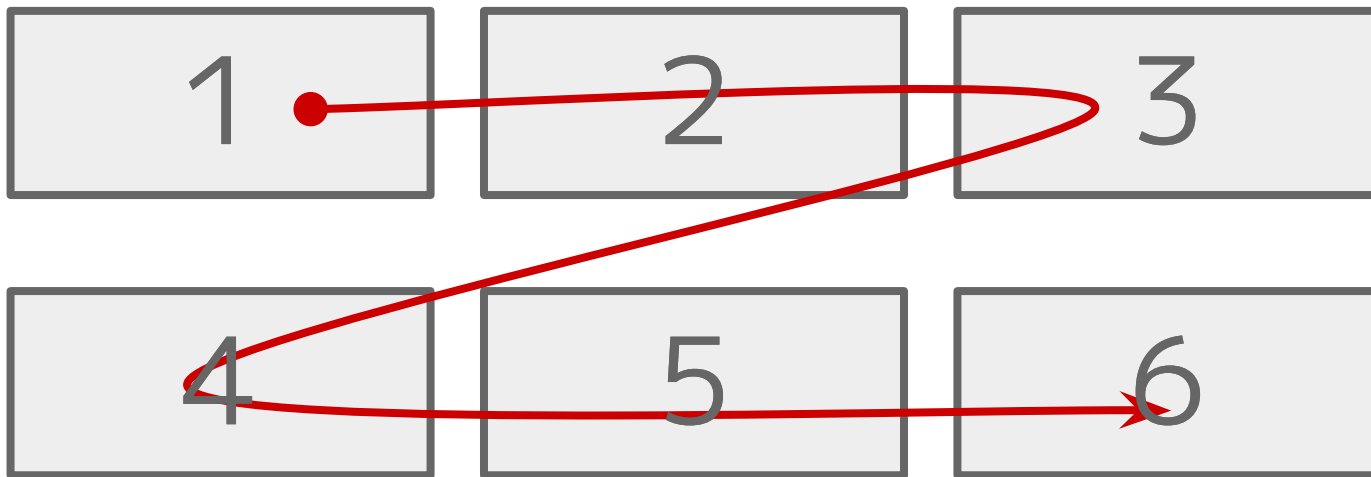
# 2行1列のグラフの1つ目
② plt.subplot(2, 1, 1)
x = np.linspace(-10, 10, 100)
plt.plot(x, np.sin(x))

# 2行1列のグラフの2つ目
③ plt.subplot(2, 1, 2)
y = np.linspace(-10, 10, 1000)
plt.plot(y, np.sin(2*y))

plt.grid(True)
```



```
plt.subplot(2, 3, n)
```





【Matplotlibの書き方】(pyplotも同じ/一応Seabornもこの書き方でもOK)

```
plt.scatter(x = df['sepal_length'], y = df['petal_width'])
```

x軸のデータの指定

y軸のデータの指定

【Seabornの書き方】

```
sns.scatterplot(data=df, x='sepal_length', y='petal_width')
```

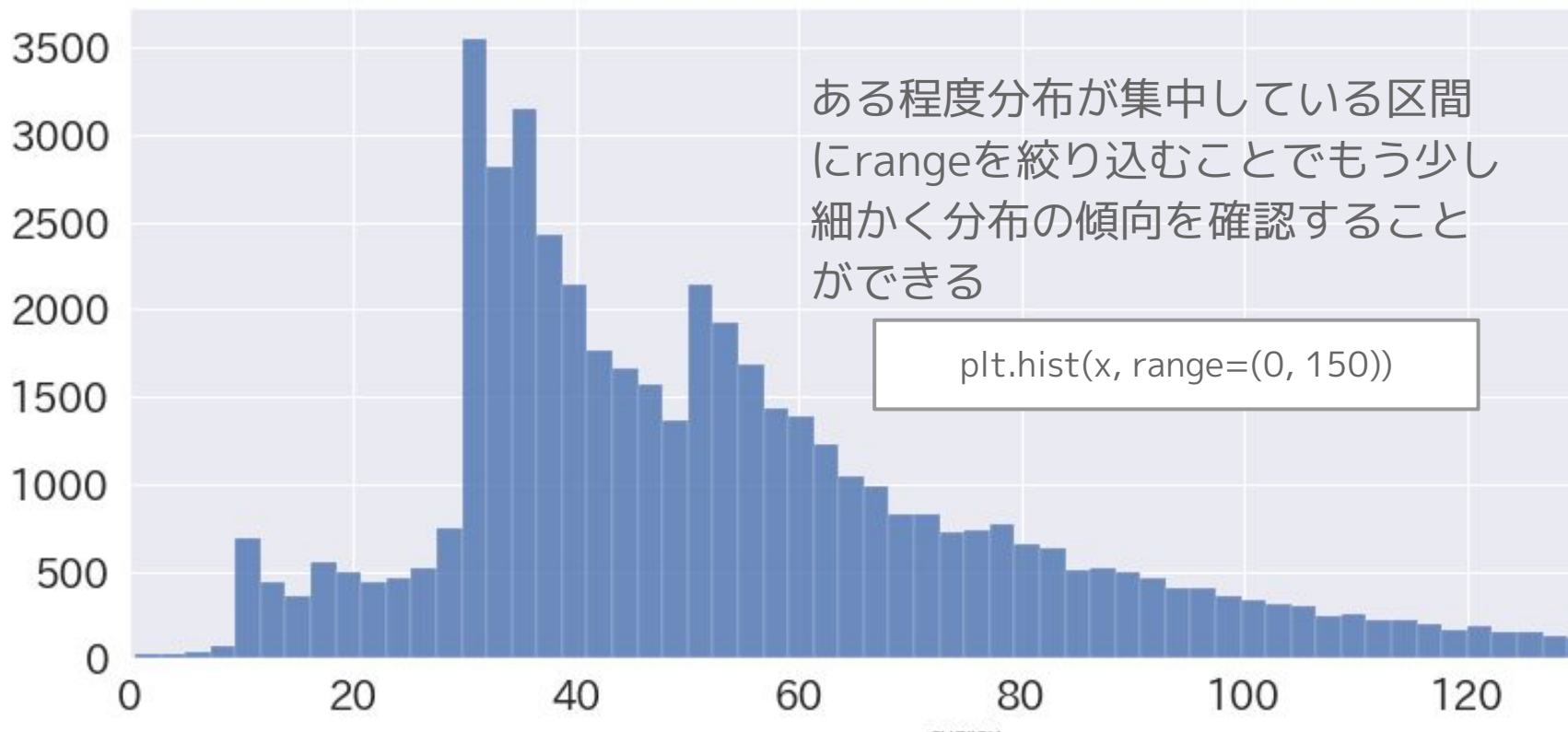
データセットの指定

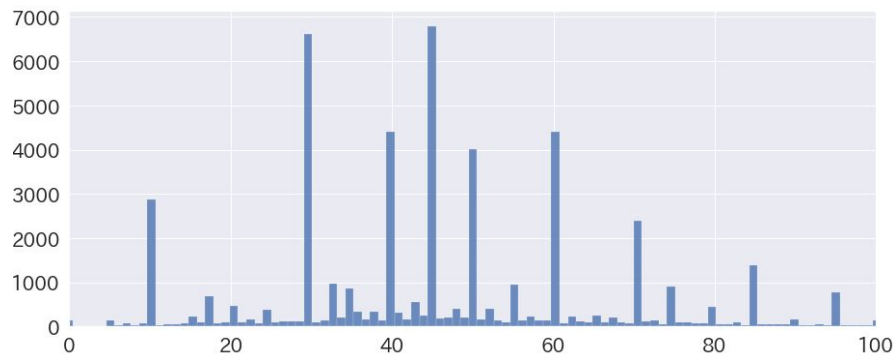
x軸のカラム名の指定

y軸のカラム名の指定

SeabornでもMatplotlibと同じ書き方が一応できるが、引数「data」にデータセットを指定したうえで、引数x・yにはカラム名だけを書くスタイルの方がSeabornではより一般的な書き方

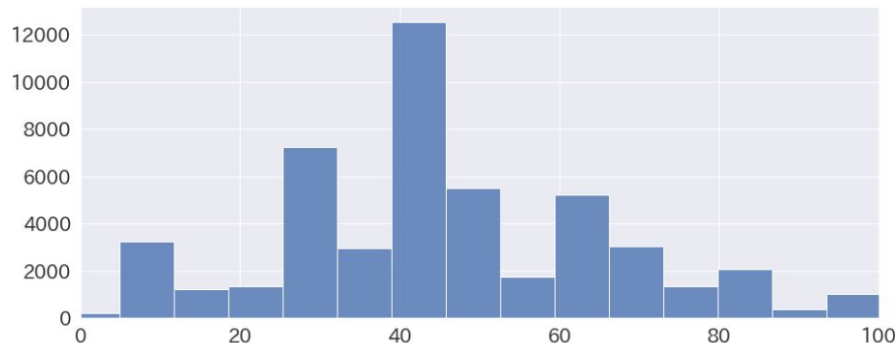






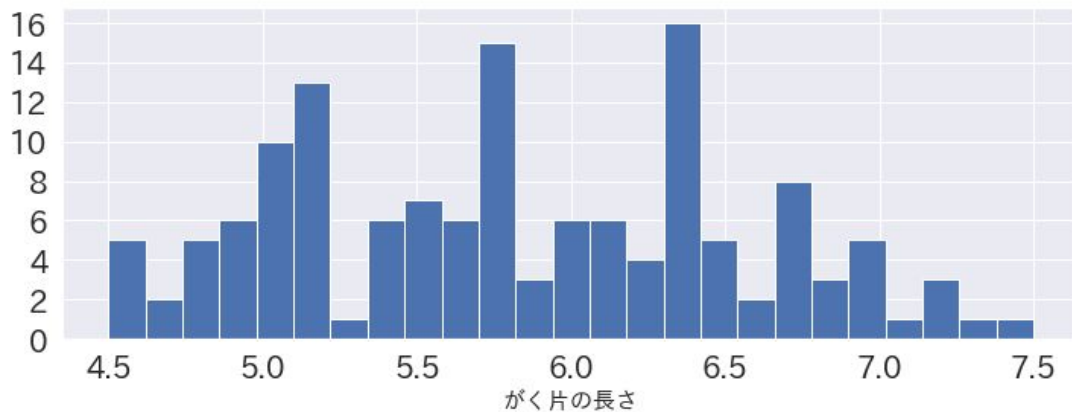
ビンを細かく切った場合

- 局所的に分布の集中している区間を発見できる場合がある
- 全体像としての傾向が掴みにくなる



ビンを大まかに切った場合

- 全体的な傾向を把握しやすい
- 局所的に生じている特異的な分布は見えなくなる



引数「bins」に整数型の値を渡して、区間をいくつで切るか指定する。

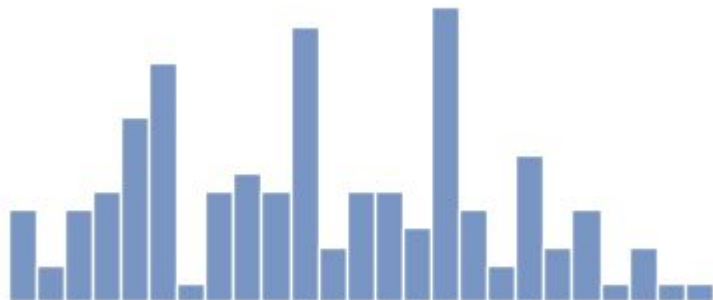
`plt.hist(x, bins={ビン切りする数})`で指定する。

【例】

`plt.hist(x, bins=25)`

ヒストグラムをカテゴリカルな変数で色分けする

```
sns.histplot(  
    data=df,  
    x='sepal_length',  
    bins=25,  
    binrange=(4.5, 7.5),  
)
```

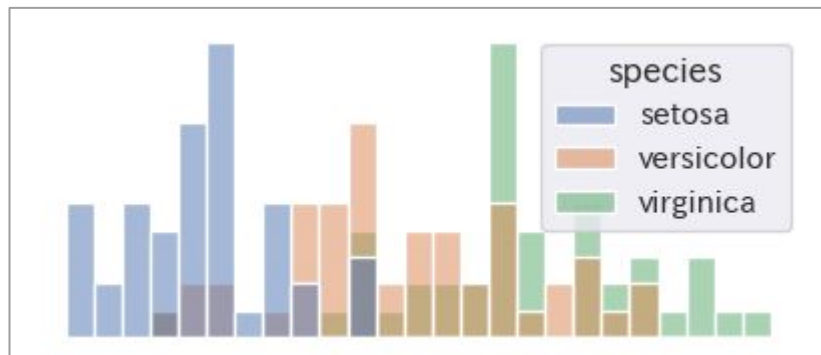


```
sns.histplot(  
    data=df,  
    x='sepal_length',  
    bins=25,  
    binrange=(4.5, 7.5),  
    hue='species', }ここを追加  
)
```

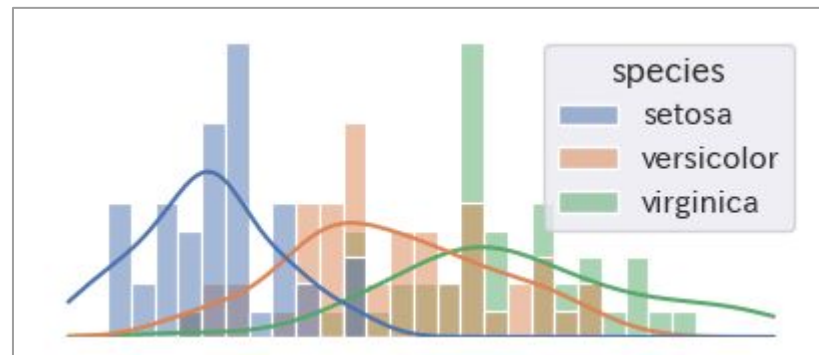


ヒストグラムにkdeプロットを重ね描きする

```
sns.histplot(  
    data=df,  
    x='sepal_length',  
    bins=25,  
    binrange=(4.5, 7.5),  
    hue='species',  
)
```



```
sns.histplot(  
    data=df,  
    x='sepal_length',  
    bins=25,  
    binrange=(4.5, 7.5),  
    hue='species',  
    kde=True, }ここを追加  
)
```



棒グラフをつくる関数 plt.bar()

plt.bar()で棒グラフを指定

引数のalignは棒グラフをX軸のどこに合わせるかを指定するもの

'center': バーの中心をxtickに合わせる

'edge': バーの左端をxtickに合わせる

※バーの右端をxtickに合わせる場合は'edge'を選択した上で、widthに負の数を指定する。

plt.xticks(), plt.yticks()

x,yの各軸の目盛りのパラメータを指定

第一引数に目盛りを入れる位置の値のリスト、

第二引数に目盛りラベルをリストで指定



表示するデータ

x = [1, 2, 3]

y = [10, 1, 4]

①

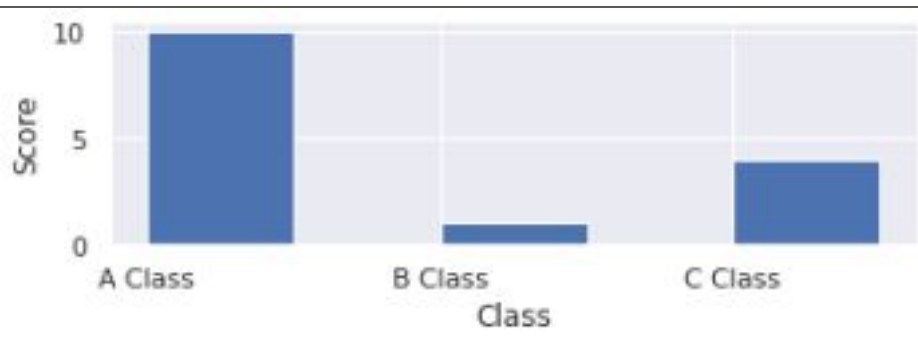
②

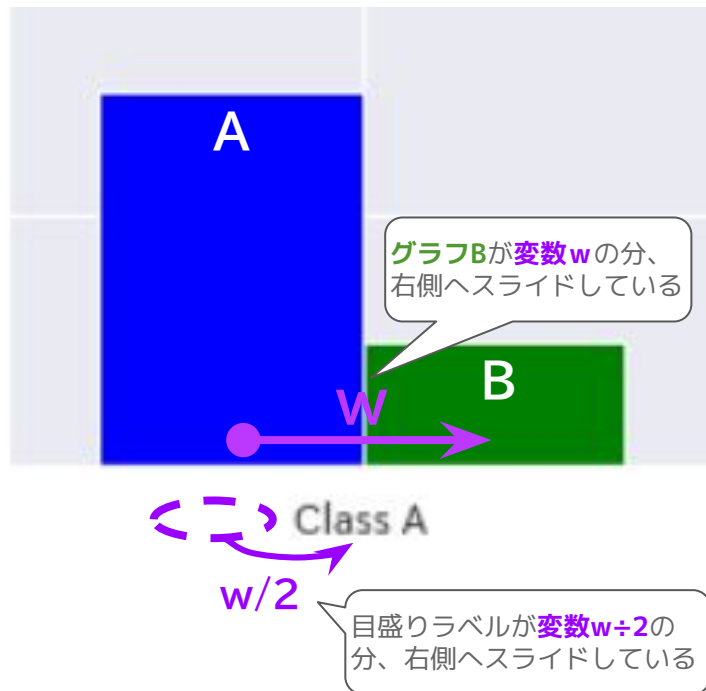
plt.bar(x, y, align='edge', width = 0.5)

③

棒グラフそれぞれのラベル

plt.xticks(x, ['A Class', 'B Class', 'C Class'])





$w = 0.4$

①「棒グラフの太さ」に後で指定したい値を先に変数 w に入れておく（コードをシンプルにするため）

②グラフAは通常通りに指定する

```
plt.bar(x, y1, width = w)
```

x軸方向の位置 y軸方向の位置

③グラフBは「x軸方向の位置」として「グラフAの太さ」である変数 w を足した値を指定する

```
plt.bar(x + w, y2, width = w)
```

x軸方向の位置 y軸方向の位置 棒の太さ

④x軸の目盛りラベルの「x軸方向の位置」として「グラフAの太さ」である変数 w の半分を足した値を指定する

```
plt.xticks(x + w / 2, ['Class A', 'Class B'],
```

x軸方向の位置



```
p1 = plt.bar( x, height1,
```

x軸方向の位置

棒グラフの高さ

p1の「棒グラフの高さ」の値をそのまま
p2の「y軸方向の高さ」に指定している。

```
p2 = plt.bar( x, height2, bottom = height1,
```

x軸方向の位置

棒グラフの高さ

y軸方向の位置

引数「**bottom**」

棒グラフのy軸方向の高さを指定する引数。
渡した値の分、お尻が持ち上がるイメージ。

【使い方】

表示をしたい要素の順に値を入力したリストをそれぞれの引数に渡して使う。

【主な引数】

x

各要素の値

label

各要素に表示する文字列

colors

各要素の色

explode

各要素を中心からどの程度離して表示するか

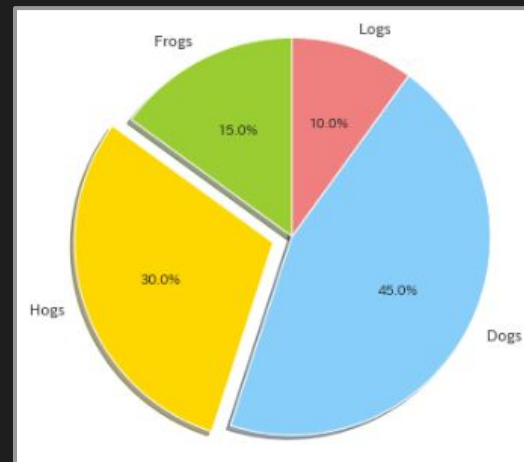
startangle

要素を並べる起点の位置を角度で指定する（「0」に指定すると3時の位置が起点になる）

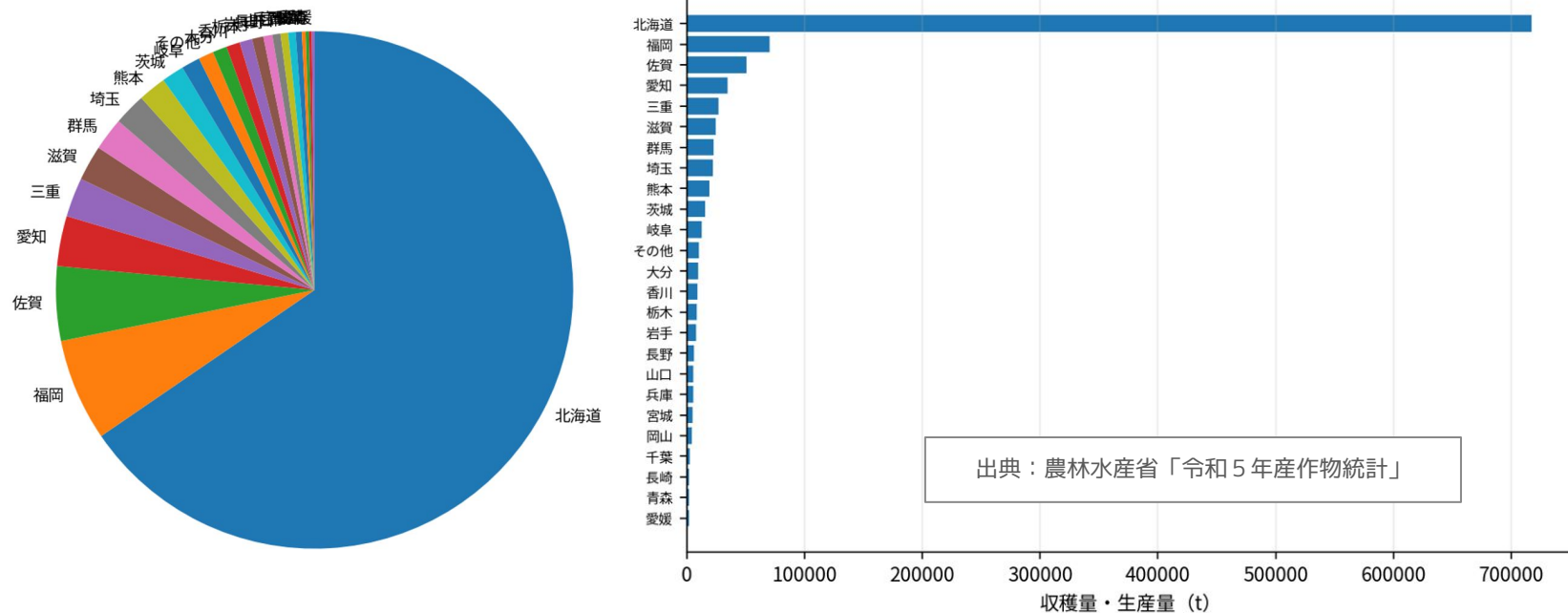
```
labels = [ 'Frogs', 'Hogs', 'Dogs', 'Logs' ]
sizes = [ 15, 30, 45, 10 ]
colors = [ 'yellowgreen', 'gold', 'lightskyblue', 'lightcoral' ]
explode = ( 0, 0.1, 0, 0 )
```

グラフを表示

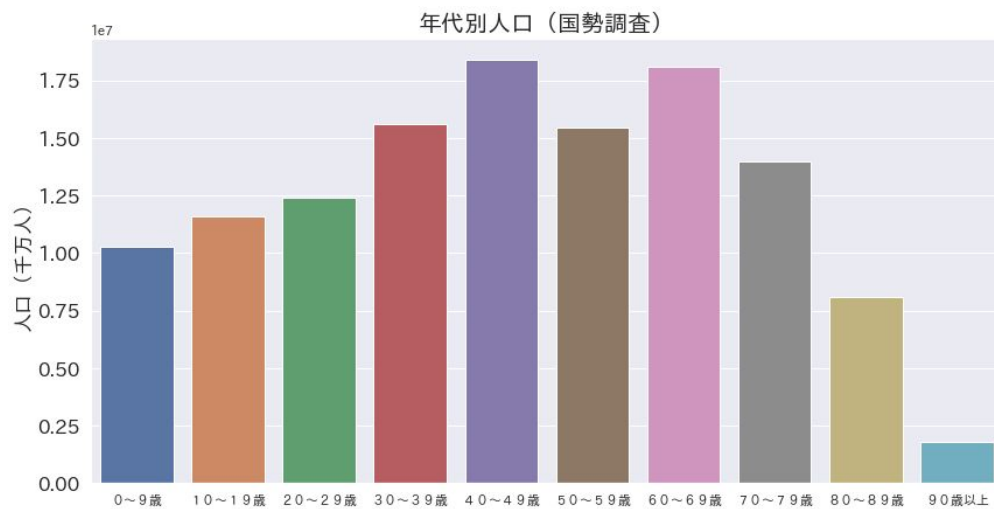
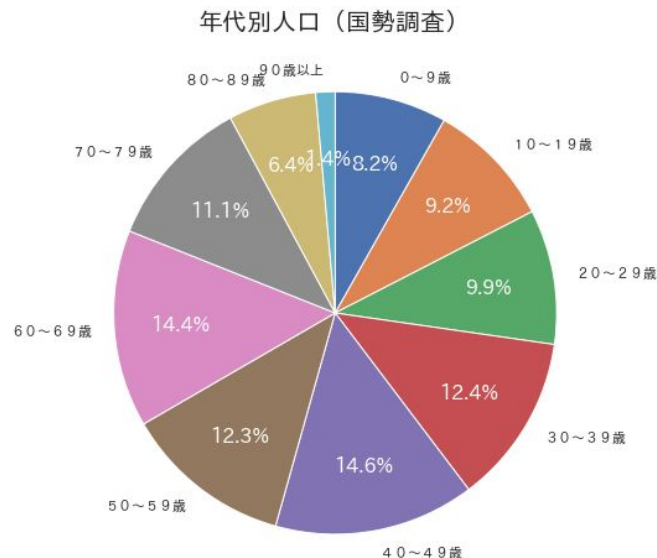
```
plt.pie(x = sizes,
        explode = explode,
        labels = labels,
        colors = colors,
        autopct = '%1.1f%%',
        shadow = True,
        startangle = 90)
```



令和 5 年産小麦収穫量の例



- 支配的シェアが存在することを目立たせる効果はある
- ただし棒グラフでも同じことは把握できる
- 円グラフは要素数が多いとラベルが潰れて調整に手間がかかる



出典：総務省統計局「令和2年国勢調査」人口等基本集計（2020年10月1日現在）を基に作成

- 支配的なシェアの要素がない場合は漫然とした印象のグラフになりやすい
- 棒グラフに比べて円グラフでは要素間の差が直感的に把握しにくい

2つの教材を用いますが、データ可視化と単回帰分析に必要な話だけにフォーカスして解説します

Pythonによるデータ可視化の基礎 (Matplotlib) .ipynb

- 1.1 データの可視化
- 1.2 データ可視化の基礎
- Appendix: EDAを自動化する様々なライブラリ

統計・確率.ipynb ---> こちらの教材もあくまでデータ可視化の話のみにフォーカスします

- 2.3.4 要約統計量とパーセンタイル値
- 2.3.5 箱ひげ図

→ 箱ひげ図の理解

- 2.3.7 散布図と相関係数

→ ヒートマップの理解

- 2.4 単回帰分析

→ ※このトピックだけ少し別テーマ（機械学習の入り口の話）

上記以外の箇所はGCI修了後に更なるスキルアップを目指す際に役立ててください

直感的な定義：小さい値から数えて「 $n\%$ 目の値はいくつか」を示す値

※より厳密には下位 $n\%$ のデータがその値以下になるような値を指す

もし100名の学生がいるクラスでの試験結果が下記であったとき・・・

順位	名前	点数
1位	Aさん	100
⋮	⋮	⋮
25位	Zさん	80
⋮	⋮	⋮
50位	Dさん	70
⋮	⋮	⋮
75位	Fさん	50
⋮	⋮	⋮
100位	Jさん	30

→ 75パーセンタイル (第3四分位点)

→ 50パーセンタイル (第2四分位点) = **中央値** (Q2)

→ 25パーセンタイル (第1四分位点)

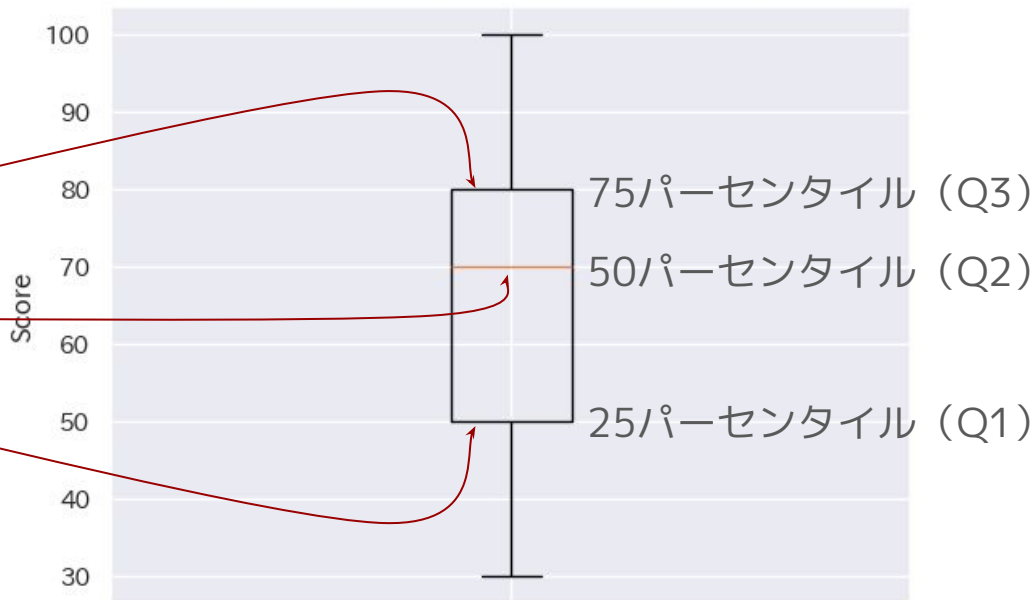
第1四分位点と第3四分位点の区間を **四分位範囲** という
(IQR)

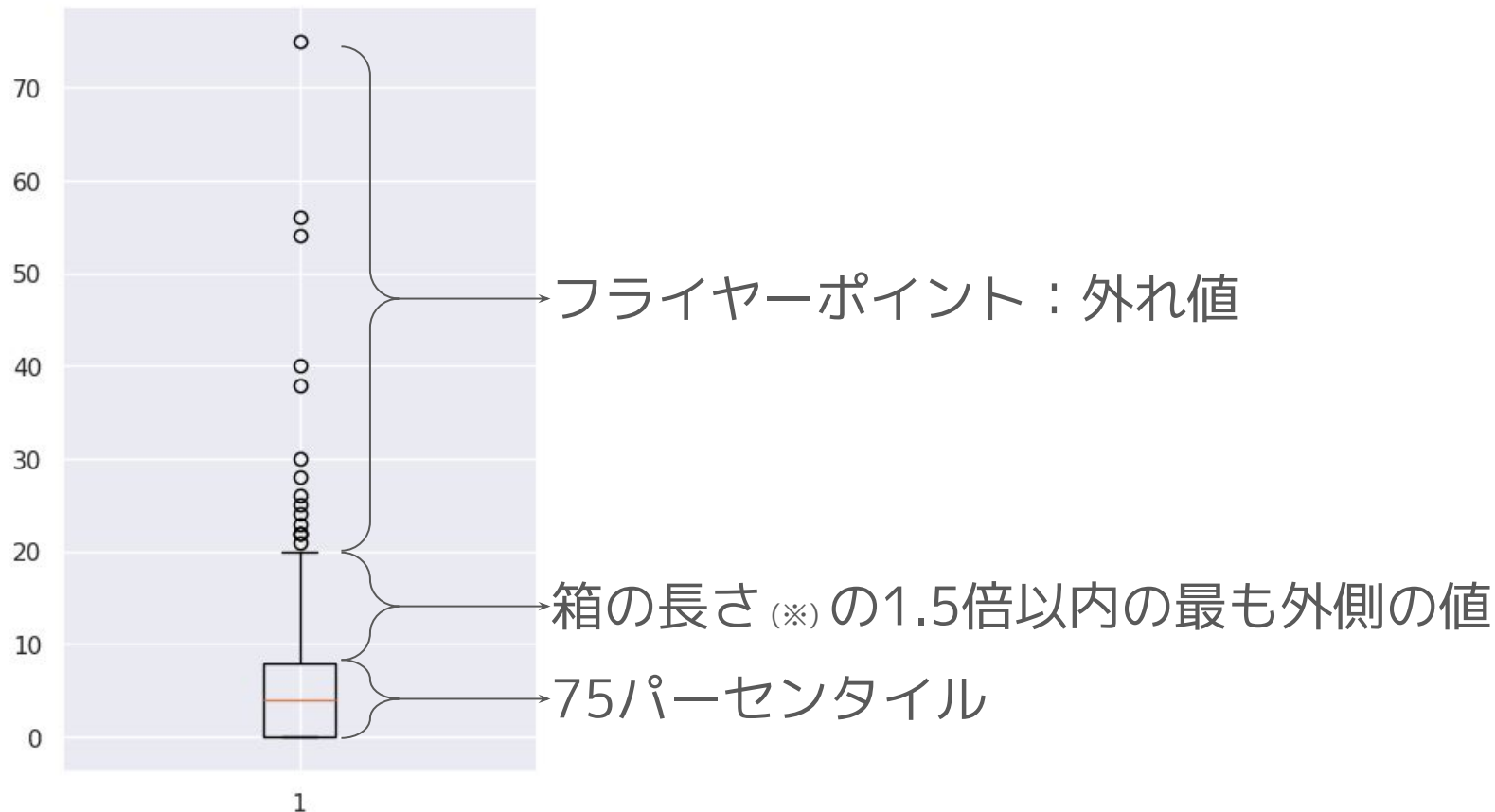
直感的な定義：小さい値から数えて「 $n\%$ 目の値はいくつか」を示す値

※より厳密には下位 $n\%$ のデータがその値以下になるような値を指す

もし100名の学生がいるクラスでの試験結果が下記であったとき・・・

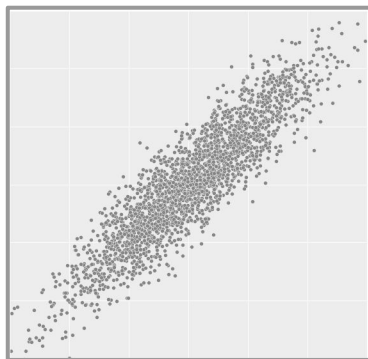
順位	名前	点数
1位	Aさん	100
⋮	⋮	⋮
25位	Zさん	80
⋮	⋮	⋮
50位	Dさん	70
⋮	⋮	⋮
75位	Fさん	50
⋮	⋮	⋮
100位	Jさん	30



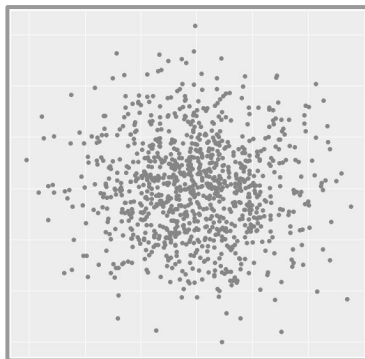


※ 箱の長さ = 第3四分位点 - 第1四分位点

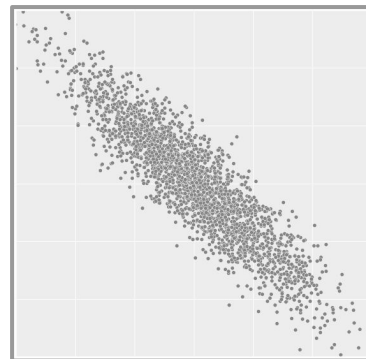
- $-1 \sim 1$ までの値を取る
- 正の相関が強いと 1に近づく
- 負の相関が強いと -1 に近づく
- 0に近いほど相関が弱い
- 一方の値が変化すると他方の値も変化するような連動性を表現する係数
- 因果関係があるかないかは定義外



正の相関



相関がない



負の相関

話が多かったと感じた人は、まずはこれが頭に残っていればOK GCI

気持ちの持ちかた

- ① **最小限にはたった一行でOK**なので気楽にやる
- ② 作り込まなくてOK。それよりも**手数を重視**！

散布図

```
plt.scatter(x, y)
```

2つの変数間の関係をみたいとき

箱ひげ図

```
plt.boxplot(x)
```

データのばらつきや外れ値をみたいとき

折れ線グラフ

```
plt.plot(x, y)
```

時間の変化や流れをみたいとき

ヒストグラム

```
plt.hist(x)
```

値の分布の形をみたいとき

棒グラフ

```
plt.bar(x, y)
```

カテゴリごとの値を比べたいとき

ヒートマップ

```
sns.heatmap(df.corr(numeric_only=True))
```

値の強さやパターンを色でみたいとき

データ可視化

- 数値を**グラフ**で表す
- そこから考察
- 探索的にグラフを改善



線形単回帰分析

- 数値から**未来を予測**
- 実測値の分布に近似する直線を引く



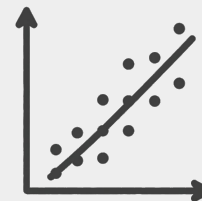
データ可視化

- 数値を**グラフ**で表す
- そこから考察
- 探索的にグラフを改善



線形単回帰分析

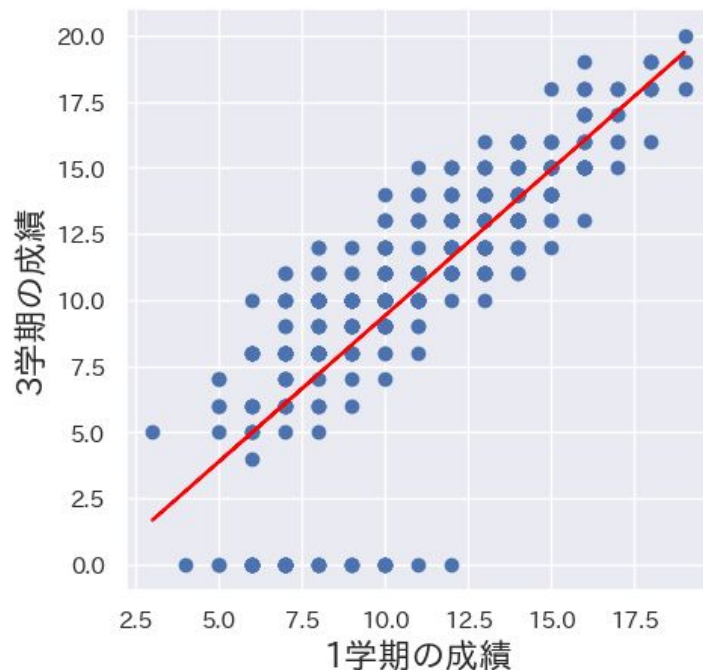
- 数値から**未来を予測**
- 実測値の分布に近似する直線を引く



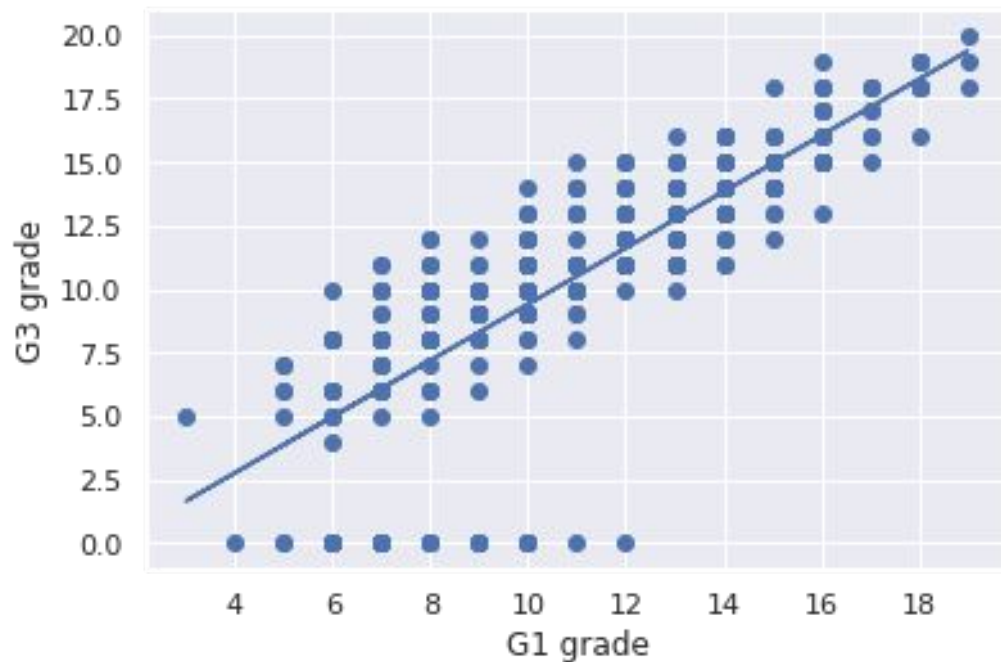
2つのデータの関係を**近似する直線**を引き、これを使って**予測すること**

【1学期の成績】の値が手に入れば【3学期の成績】を予測できるようになりたいな…
説明変数 **目的変数**

- ・ 手に入りやすいデータを使って、手に入りにくいデータを予測したいときに活躍する
- ・ 例えば…
 - 過去のデータから未来のデータを予測
 - 経済指標から商品需要量を予測 など



線形単回帰の直線は一次関数の方程式で表せる



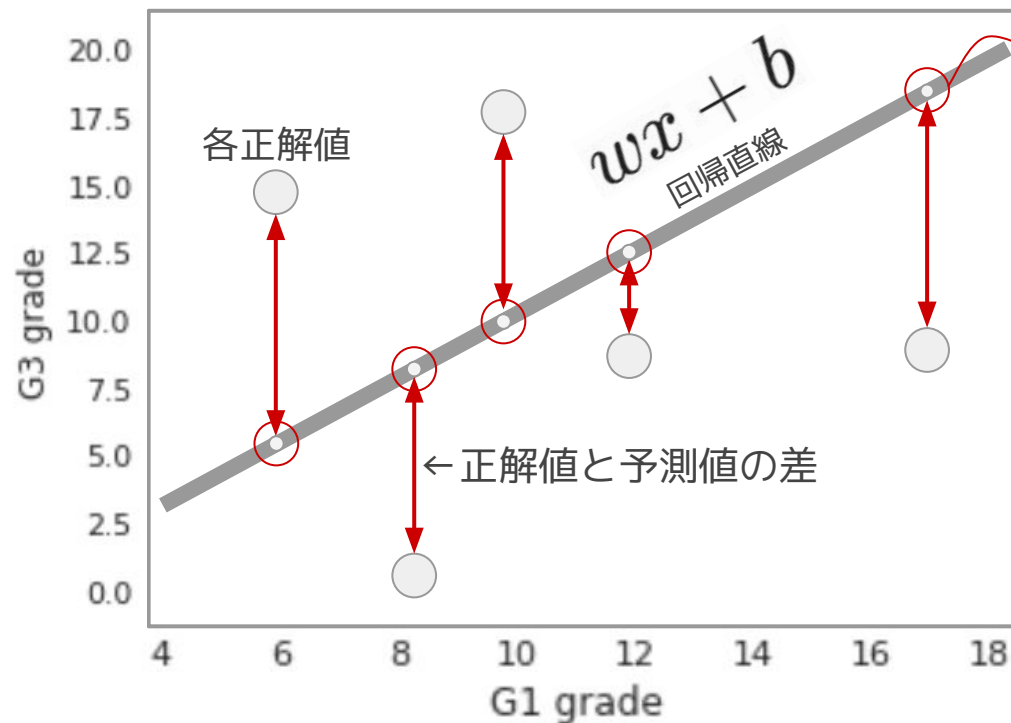
$$y = wx + b$$

目的変数 y 説明変数 x
回帰係数(傾き) w 切片 b



中学数学の復習

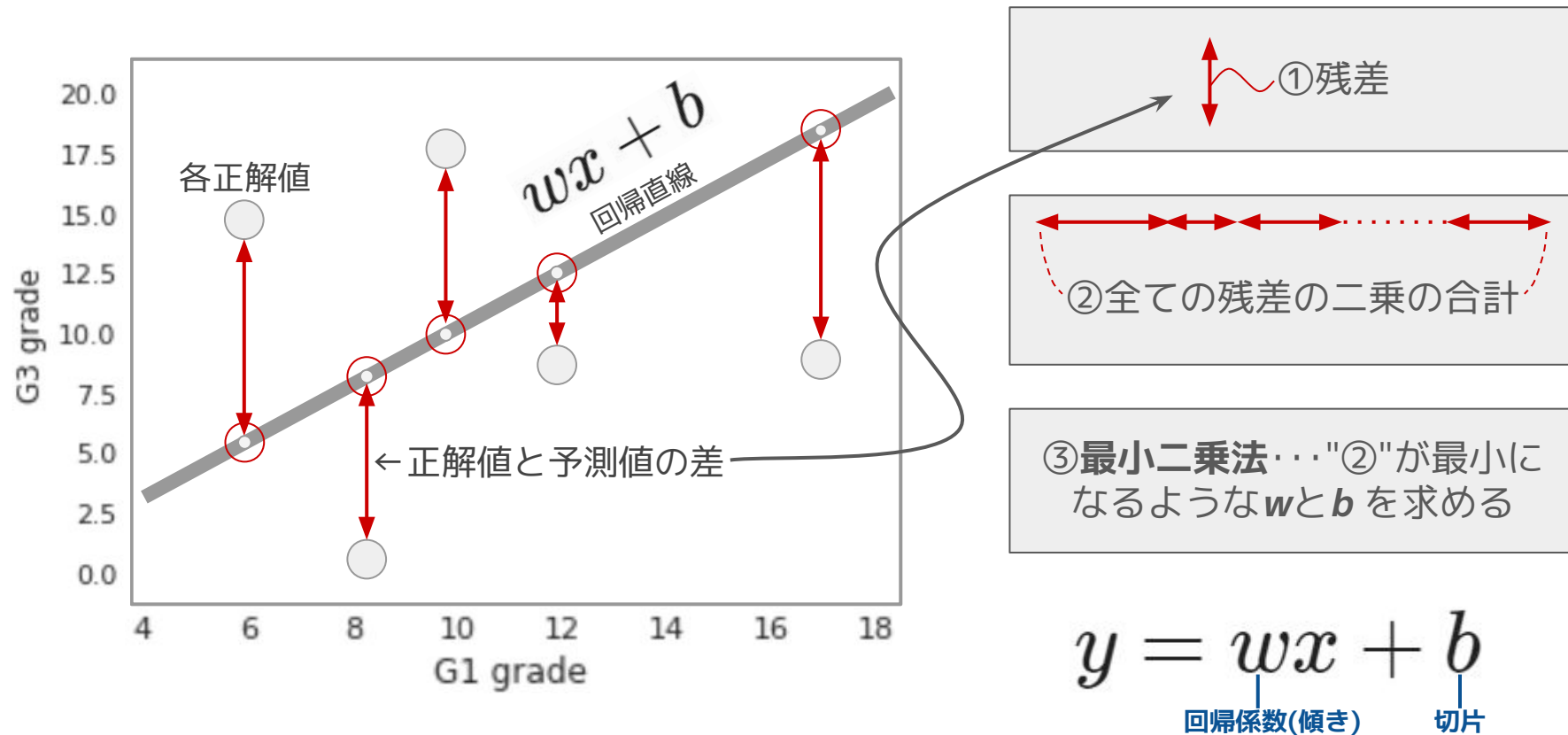
切片・・・直線がy軸と交わる位置の値



回帰直線によって求められる予測値
※ w と b が定まっているときに x を決めると y が予測値として算出される

$$y = wx + b$$

目的変数 y 説明変数 x 切片 b
回帰係数(傾き) w



- 線形回帰モデルなどの予測モデルを提供
- 評価用の関数など関連ツールも備える



Copyright © The scikit-learn developers

<https://github.com/scikit-learn/scikit-learn/blob/main/doc/logo/scikit-learn-logo.svg>

- matplotlibと同じくライブラリからimportして利用

```
# ライブラリからインポート
from sklearn import linear_model

# 線形単回帰モデルを呼び出す
reg = linear_model.LinearRegression ()
```

予測モデルが格納される

モデルの呼び出し



Step1

予測モデルの**呼び出し**

線形単回帰モデルを呼び出す

```
reg = linear_model.LinearRegression()
```



Step2

fitメソッドで**学習**
説明変数と目的変数を渡す

モデルに過去データの学習をさせる

```
reg.fit(X, Y)
```



Step3

predictメソッドで**予測**

新たに予測したい説明変数を渡して予測させる

```
reg.predict(X_new)
```



Step1

予測モデルの**呼び出し**

線形単回帰モデルを呼び出す

```
reg = linear_model.LinearRegression()
```



Step2

fitメソッドで**学習**
説明変数と目的変数を渡す

モデルに過去データの学習をさせる

```
reg.fit(X, Y)
```

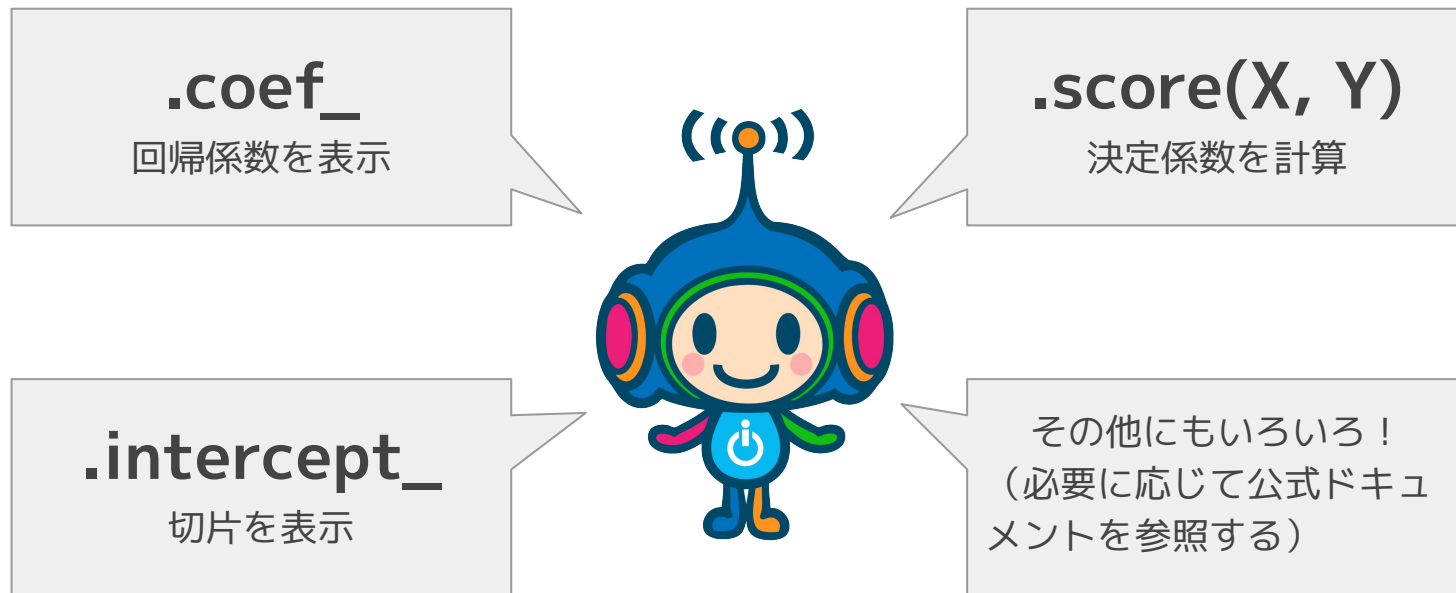


Step3

predictメソッドで**予測**

新たに予測したい説明変数を渡して予測させる

```
reg.predict(X_new)
```



Scikit-learnは各ライブラリの中でも特に公式ドキュメントが見やすい。
→積極的に公式の情報を参照するのがおすすめ。

CSVをDataFrameで読み込む

```
df = pd.read_csv("csvを保存したファイルパス")
```

説明変数と目的変数を用意する

```
x = df[["説明変数"]] # 説明変数を抽出
```

```
y = df["目的変数"] # 目的変数を抽出
```

モデルをインポート

```
from sklearn.linear_model import LinearRegression
```

予測モデルの3ステップ！

```
model = LinearRegression() # モデル呼び出し
```

```
model.fit(X_train, y_train) # 学習
```

```
y_pred = model.predict(X_test) # 予測
```


チートシートとは？・・・作業時に手元において参照するのに適する資料
多くのものが配布されているので手元に置いておくと便利

<https://matplotlib.org/cheatsheets/>

公式版：Matplotlib Development Team, Matplotlib Documentation,



検索すると非公式でも優れたものが多数ヒットします

